Ride-Hailing Surge Pricing Engine: A Concurrent Dynamic-Pricing System Using Greedy and Dynamic Programming Algorithms with AsyncIO

COMPUTATIONAL THINKING AND PROGRAMMING report
Submitted in partial fulfillment of the requirements
for the award of degree of

MASTER OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

VUNDRU VENKATA SAI DURGESH 2601050109

Under the supervision of

Dr Raju Anitha

Professor

Department of Computer Science and Engineering
Koneru Lakshmaiah Education Foundation (Deemed
to be University estd., u/s 3 of UGC Act 1956)
Green Fields, Vaddeswaram-522502, Guntur (Dist.), Andhra Pradesh, India

KONERU LAKSHMAIAH EDUCATION FOUNDATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



DECLARATION

The Computational Thinking and Programming Report entitled "Ride-Hailing Surge Pricing Engine" is a record of my own bona fide work submitted in partial fulfilment of the requirements for the award of the Master of Technology degree in Computer Science and Engineering to K L Deemed to be University during the academic year 2026–2027.

VUNDRU VENKATA SAI DURGESH
Student ID: 2601050109

KONERU LAKSHMAIAH EDUCATION FOUNDATION

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the report entitled "Ride-Hailing Surge Pricing Engine" is being submitted by VUNDRU VENKATA SAI DURGESH (Student ID: 2601050109) in partial fulfilment of the requirements for the award of the Master of Technology degree in Computer Science and Engineering at K L Deemed to be University during the academic year 2026–2027.

Dr Raju Anitha
Professor

ACKNOWLEDGEMENT

I express my sincere gratitude to my project guide, Dr Raju Anitha, for the valuable guidance, constant encouragement and constructive suggestions that helped me to complete this project successfully.

I thank the Head of the Department of Computer Science and Engineering and the Principal of K L Deemed to be University for providing the facilities and a supportive environment for the work.

I am also thankful to all the faculty members of the department, to the open-source communities of Python, Streamlit, Plotly, Pandas and Pytest whose tools made this project possible, and to my family and friends for their continuous support.

Vundru Venkata Sai Durgesh

ABSTRACT

Ride-hailing platforms must balance the number of ride requests (demand) against the number of available drivers (supply). Surge pricing raises fares in zones where demand exceeds supply, encouraging drivers to move to those zones and encouraging less urgent riders to wait. A direct mapping from the instantaneous demand/supply ratio to a price is simple but unstable: small fluctuations change the price, and short bursts of demand cause sudden spikes.

This project presents the Ride-Hailing Surge Pricing Engine, a concurrent engine written in Python 3.12. For each of five zones - Vijayawada, Hyderabad, Bangalore, Chennai and Mumbai - the engine calculates the demand/supply ratio and selects a pricing tier of 1.0x, 1.2x, 1.5x, 2.0x or 2.5x using a greedy algorithm. A dynamic-programming layer stores solved (zone, demand, supply) states in a memo table and applies a smoothing recurrence over the previous multiplier of the zone, limiting how fast prices change. The zones are processed simultaneously with AsyncIO, using one producer task per zone, an asynchronous queue and a pool of worker tasks. The system is built with object-oriented principles and includes a Streamlit dashboard and a Pytest suite of 68 test cases.

In a simulated experiment of 300 ticks for each of the five zones, smoothing reduced the average tick-to-tick price change by about 22 percent and eliminated all 59 jumps of 0.5x or more, without changing the average multiplier. Processing five zones concurrently took 0.53 seconds against 2.5 seconds for sequential processing. The project demonstrates how the greedy strategy, dynamic programming, asynchronous programming and object-oriented design can be combined into a transparent and testable pricing system.

Keywords: surge pricing, dynamic pricing, greedy algorithm, dynamic programming, memoisation, AsyncIO, ride-hailing, Streamlit.

TABLE OF CONTENTS

Declaration....2
Certificate....3
Acknowledgement....4
Abstract....5
1. Introduction....7
2. Problem Statement....9
3. Objectives....10
4. Literature Survey....11
5. Existing System....13
6. Proposed System....14
7. System Architecture....16
8. UML Class Diagram....18
9. Activity Diagram....19
10. Algorithms....20
11. Implementation Details....25
12. Screenshots....27
13. Test Cases....30
14. Results....32
15. Advantages....35
16. Future Enhancements....36
17. Conclusion....37
References....38

1. Introduction

1.1 Background

Ride-hailing platforms such as Uber, Ola and Rapido connect riders with nearby drivers through a mobile application. A fundamental difficulty of such two-sided marketplaces is that demand and supply are never perfectly balanced. During office closing hours, heavy rain, festivals or sporting events the number of ride requests in a locality can grow several times within a few minutes, while the number of drivers who are free to serve them does not change at the same speed. Riders then wait for a long time or fail to find any vehicle at all, and drivers are not given any reason to move towards the busy locality.

Surge pricing (also called dynamic pricing) is the mechanism most platforms use to restore balance. When the ratio of demand to supply in a zone rises, the platform multiplies the normal fare by a factor greater than one. The higher fare has two intended effects: riders whose need is less urgent postpone or cancel their trip, and drivers from neighbouring areas are attracted to the zone. The multiplier is therefore a control signal whose purpose is to bring the marketplace back to equilibrium.

Designing the software that produces this signal is an interesting engineering problem. The calculation must be performed for many zones at the same time, it must react to real-time data, it must be fast, and the output must be stable enough that a customer does not see a fare jump from 1.0x to 2.5x and back within one minute. This project, the Ride-Hailing Surge Pricing Engine, addresses these requirements with a compact but complete implementation in Python.

1.2 Motivation

The project brings together several topics that are normally studied separately in a computer science curriculum: algorithm design (greedy strategy and dynamic programming), concurrent programming (Python's AsyncIO), object-oriented design, automated testing and data visualisation. A surge-pricing engine is a realistic setting in which all of them are needed at once, which makes it a suitable final-year project. The result is not only an academic exercise: the architecture (producers, a queue, a worker pool and a thread-safe results store) is the same one used in many production streaming systems.

1.3 Scope of the Project

The system simulates five city zones - Vijayawada, Hyderabad, Bangalore, Chennai and Mumbai - each with its own fleet size, demand level and daily demand pattern. For every zone

the engine continuously reads the number of pending ride requests (demand) and the number of available drivers (supply), computes a surge multiplier, converts it into a fare and shows the result on a live dashboard. The project does not connect to a real ride-hailing back end; demand and supply are generated by a simulator so that the algorithms can be observed, tested and reproduced in a classroom environment.

1.4 Organisation of the Report

Chapter 2 states the problem and Chapter 3 lists the objectives. Chapters 4 and 5 review the literature and the existing systems. Chapter 6 describes the proposed system, Chapters 7 to 9 present the architecture, the UML class diagram and the activity diagram, and Chapter 10 explains the algorithms in detail. Chapter 11 documents the implementation, Chapter 12 gives screenshot placeholders for the dashboard, Chapter 13 lists the test cases and Chapter 14 reports the results. Advantages, future enhancements, the conclusion and references follow.

2. Problem Statement

To design and implement a concurrent ride-hailing surge-pricing engine that dynamically adjusts fare multipliers according to real-time supply and demand conditions in several city zones, while maintaining pricing stability, driver availability, customer affordability and configurable surge constraints.

2.1 Detailed Problem Description

A naive implementation that computes a multiplier directly from the instantaneous demand/supply ratio suffers from four problems.

- Instability. Demand and supply are noisy. A small random change of the ratio near a tier boundary changes the multiplier, and the customer sees a different price every few seconds.
- Sudden spikes. A short burst of requests (for example a concert ending) can push the multiplier to its maximum in a single step, which damages customer trust and affordability.
- Redundant work. The same (zone, demand, supply) situation is evaluated again and again without reusing earlier results.
- Sequential processing. If zones are processed one after another, the price of the last zone is always older than the price of the first one.

The system must therefore combine a simple and explainable pricing rule with a mechanism that remembers history, and it must process all zones concurrently.

2.2 Constraints

- The multiplier must stay within configurable minimum and maximum limits (default 1.0x to 2.5x).
- The base fare and the tariff must be configurable without changing the code of the pricing algorithm.
- The pricing rules must be deterministic and explainable: for the same input the same tier must be selected.
- The solution must run on an ordinary student laptop with Python 3.12 and no database or message broker.

3. Objectives

The main objective is to build a complete, executable and well-tested surge-pricing engine. The specific objectives are:

- To calculate the demand/supply ratio of every zone and convert the imbalance into a surge multiplier within configurable minimum and maximum limits.
- To implement a greedy algorithm that selects a pricing tier immediately from the current ratio using the rules 1.0x, 1.2x, 1.5x, 2.0x and 2.5x.
- To implement dynamic programming with a memo table of (zone, demand, supply) states and a smoothing recurrence that removes sudden fare fluctuations.
- To process five zones simultaneously using AsyncIO tasks, an asynchronous queue and a pool of worker tasks.
- To apply object-oriented principles - encapsulation, composition, inheritance and dataclasses - in the classes RideRequest, Driver, Zone, DemandSimulator, SurgeCalculator, PricingEngine and FareCalculator.
- To provide a Streamlit dashboard that displays available drivers, ride requests, demand/supply ratio, surge multiplier, base fare and final fare with auto-refresh, zone selection and a historical surge graph.
- To verify the system with an automated Pytest suite and to document the architecture, design and usage.

4. Literature Survey

Surge pricing has been studied extensively by economists and computer scientists. This chapter summarises the work that influenced the design of this project.

4.1 Economic Studies of Surge Pricing

Hall, Kendrick and Nosko (2015) analysed data from Uber and found that surge pricing increases the supply of drivers in a surging area and reduces the number of unfulfilled requests, which supports the use of a multiplier as an incentive mechanism. Chen and Sheldon (2016) studied the labour-market side of the problem and showed that drivers respond to higher earnings by working more hours, so the multiplier influences supply and not only demand. Cohen, Hahn, Hall, Levitt and Metcalfe (2016) used the discontinuities created by discrete multipliers to estimate consumer surplus and concluded that dynamic pricing creates substantial value for riders who need a trip the most. Castillo, Knoepfle and Weyl (2017) argued that surge pricing avoids the so-called wild-goose-chase inefficiency, in which drivers travel long distances to pick up riders and the platform effectively becomes less productive when supply is already scarce.

4.2 Analytical Models

Banerjee, Riquelme and Johari (2015) proposed a queueing-theoretic model of ride-share platforms and showed that dynamic prices can achieve most of the welfare of an ideal state-dependent policy while being robust to errors in the estimation of demand and supply. This result motivates the use of a simple ratio-based policy in our project: an explainable rule is acceptable if it is combined with sensible limits and smoothing.

4.3 Algorithmic Foundations

The greedy strategy and dynamic programming are standard techniques described by Cormen, Leiserson, Rivest and Stein. A greedy algorithm makes the locally best choice at every step and never revisits it. Dynamic programming, introduced by Bellman (1957), solves a problem by storing the solutions of overlapping sub-problems (memoisation) or by building a table of states in which each state depends on the previous one. Our engine uses a greedy rule for the immediate decision and dynamic programming for reuse and stabilisation.

4.4 Concurrency and Software Design

Python's asyncio library provides cooperative multitasking based on an event loop, coroutines and tasks. It is well suited to I/O-bound workloads with many independent activities, such as

polling many zones, because tasks switch only at await points and therefore do not need locks for most shared data. The producer-consumer pattern with a bounded queue is a classical way to decouple data generation from processing and to apply back-pressure. For the object-oriented structure we followed the principles popularised by Gamma, Helm, Johnson and Vlissides (1994), in particular programming to an interface (the PricingStrategy base class) and the observer pattern (price listeners).

4.5 Research Gap Addressed by this Project

Most published work is economic or analytical and does not provide an executable, concurrent reference implementation. This project provides a small open implementation in which the greedy rule, the dynamic-programming cache, smoothing and concurrency can be inspected, tested and modified by students.

5. Existing System

5.1 Fixed Fares

Traditional taxi and auto-rickshaw services use fixed or metered fares. The fare depends only on distance and time and never on the availability of vehicles. During peak periods this leads to long waiting times, refusals by drivers and informal bargaining.

5.2 Time-Based Peak Pricing

Some services apply a fixed peak-hour surcharge, for example a higher fare between 8 and 10 a.m. This is simple but blind to the actual situation: it overcharges on quiet peak days and fails to react to unexpected events such as rain.

5.3 Instantaneous Ratio-Based Surge

Commercial platforms compute the multiplier from live data. A simplified version of this approach maps the current ratio directly to a multiplier. Its weaknesses, which this project targets, are summarised below.

Table 5.1: Limitations of existing approaches

Approach	Strength	Limitation
Fixed / metered fare	Predictable, simple	No reaction to demand; shortages and refusals
Time-based surcharge	Easy to explain	Ignores real conditions; unfair on quiet days
Instantaneous ratio mapping	Reacts to live data	Unstable near tier boundaries; sudden spikes; repeated computation
Sequential zone processing	Simple code	Prices of later zones are stale; poor scalability

6. Proposed System

The proposed system is a layered, concurrent pricing pipeline. It keeps the transparency of a tier-based rule but adds memory (dynamic programming) and concurrency (AsyncIO).

6.1 Features

- Concurrent processing of five zones with one asynchronous task per zone.
- A greedy tier policy that is stored as data and can be reconfigured.
- A bounded LRU memo table that stores (zone, demand, supply) to multiplier results and reports hits and misses.
- A smoothing recurrence that limits how quickly a multiplier may rise or fall, with configurable alpha and maximum step.
- Configurable minimum and maximum multiplier, base fare, per-kilometre and per-minute rates, booking fee and minimum fare.
- A thread-safe price board that lets a dashboard read results while the engine is running.
- A Streamlit dashboard with auto-refresh, zone selection, a trip estimator and a historical surge graph.

6.2 Technology Stack

Table 6.1: Technology stack

Component	Technology
Language	Python 3.12
Concurrency	AsyncIO (asyncio.Task, asyncio.Queue), threading for the dashboard bridge
Dashboard	Streamlit (st.fragment auto-refresh)
Visualisation / data	Plotly, Pandas
Testing	Pytest
Documentation	Markdown

6.3 Fare Model

The fare is calculated in three steps. First the metered fare is the sum of the base fare, the distance charge and the time charge. Second the metered fare is multiplied by the surge multiplier and compared with the minimum fare. Third a fixed booking fee is added:

```
metered = base_fare + km * per_km + minutes * per_min
final   = max(metered * multiplier, minimum_fare) + booking_fee
```

With the default tariff (base fare 50, 12 per km, 1.5 per minute, booking fee 10, minimum fare 80, in Indian rupees) a 5 km, 15 minute trip costs 142.50 at 1.0x and 275.00 at 2.0x.

7. System Architecture

The architecture is a pipeline in which every stage has one responsibility. Figure 7.1 shows the stages from the real-time simulator to the fare calculator and the dashboard.

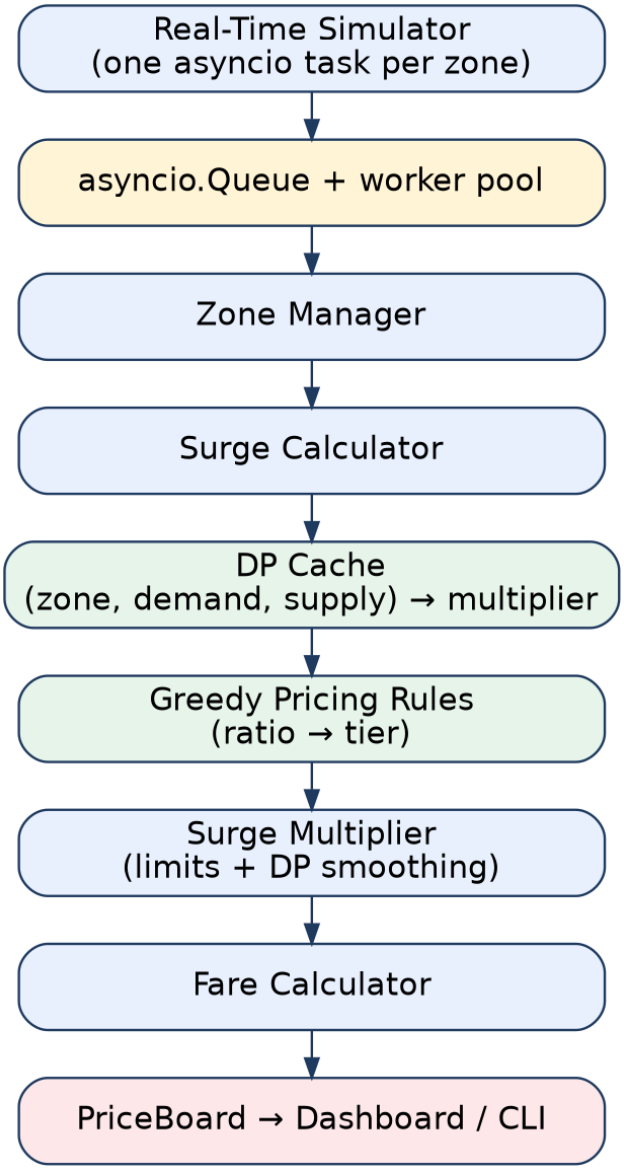


Figure 7.1: Layered architecture of the surge pricing engine

7.1 Stage Descriptions

Table 7.1: Pipeline stages

Stage	Description
Real-Time Simulator	DemandSimulator generates new ride requests, expires old ones and changes the number of available drivers. One asyncio task per zone puts a ZoneSnapshot on the queue every tick.
Queue and workers	A bounded asyncio.Queue decouples generation from pricing. A small pool of worker tasks takes snapshots and prices them.
Zone Manager	ZoneManager owns the Zone objects and gives the engine access to them.

Stage	Description
Surge Calculator	SurgeCalculator computes the ratio and orchestrates the cache, the greedy rules and the smoothing.
DP Cache	DPCache returns an already known multiplier for a (zone, demand, supply) state or asks the greedy rule to compute it.
Greedy Pricing Rules	GreedyTierStrategy maps the ratio to one of five tiers.
Surge Multiplier	The greedy value is limited to the minimum and maximum and then smoothed using the previous multiplier of the zone.
Fare Calculator	FareCalculator converts the multiplier into the final fare for a trip.
PriceBoard	A lock-protected store of the latest quote and the history of each zone, read by the dashboard and the command-line interface.

7.2 Concurrency Architecture

The engine follows the producer-consumer pattern. Five producer tasks run concurrently on one event loop. Each producer advances its own zone and awaits queue.put(), so a slow consumer slows the producers instead of allowing unlimited memory growth. Three worker tasks wait on queue.get(). Pricing a snapshot is a purely synchronous section without any await, therefore two snapshots of the same zone are never interleaved and are priced in the order in which they were produced. This property is important because the smoothing state of a zone must be updated in time order.

The dashboard runs in a separate thread of the Streamlit process. The asyncio loop is started in a daemon thread and the dashboard reads the results only through PriceBoard, whose methods are guarded by a threading.Lock. Pause, resume, stop and the tick interval are communicated through the thread-safe SimulationControl object.

8. UML Class Diagram

Figure 8.1 shows the main classes of the system and the relationships between them. Filled diamonds denote composition, the hollow diamond denotes aggregation (the calculator uses a strategy supplied from outside) and the hollow triangle denotes inheritance.

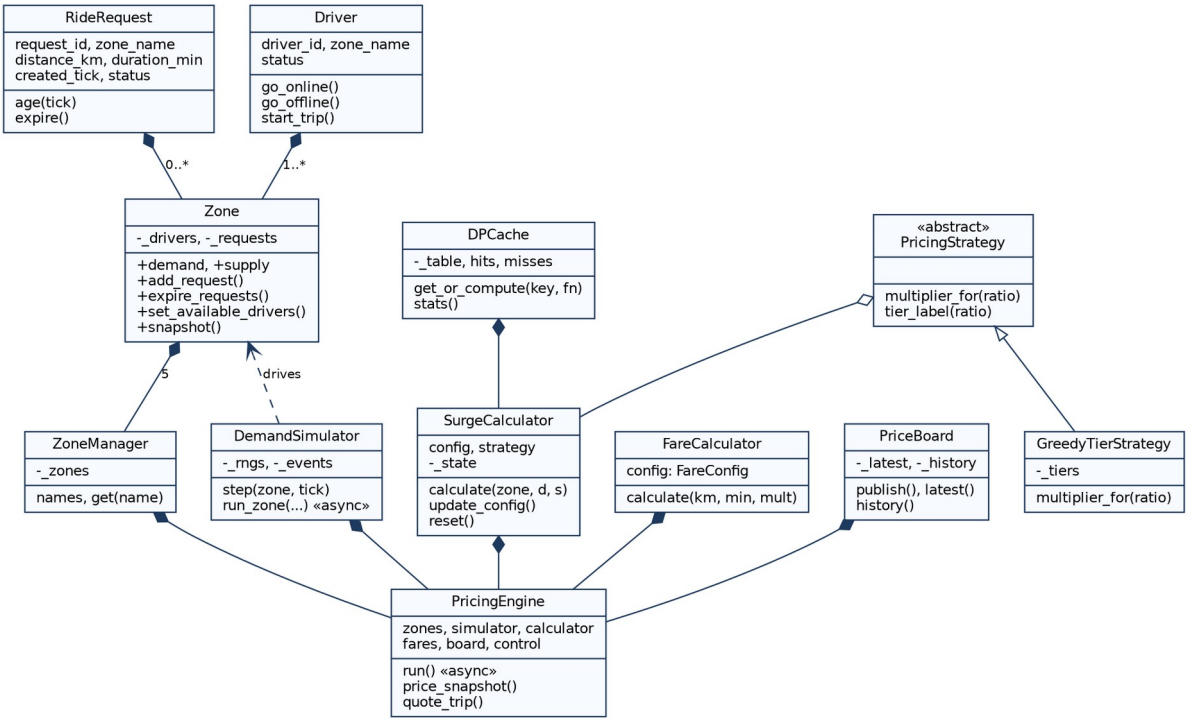


Figure 8.1: UML class diagram

8.1 Object-Oriented Principles Used

Table 8.1: OOP principles and where they appear

Principle	Application in the project
Encapsulation	Zone keeps its drivers and requests private and exposes only demand, supply and snapshot(). DPCache hides its OrderedDict. PriceBoard hides its lock.
Composition	PricingEngine is composed of ZoneManager, DemandSimulator, SurgeCalculator, FareCalculator and PriceBoard. Zone owns its Driver and RideRequest objects.
Inheritance / polymorphism	GreedyTierStrategy extends the abstract PricingStrategy. Any other strategy can be plugged into SurgeCalculator.
Dataclasses	RideRequest, Driver, ZoneProfile, ZoneSnapshot, Tier, SurgeConfig, SurgeResult, FareConfig, FareBreakdown and PriceQuote. Frozen dataclasses are used for immutable data that crosses task or thread boundaries.
Observer	PricingEngine.subscribe() lets the command-line interface print every new quote.

9. Activity Diagram

Figure 9.1 shows the flow of activity for one snapshot, from its creation by a zone task to the publication of the quote. The diamond nodes are decisions: whether the state is already stored in the DP cache and whether a stop has been requested.

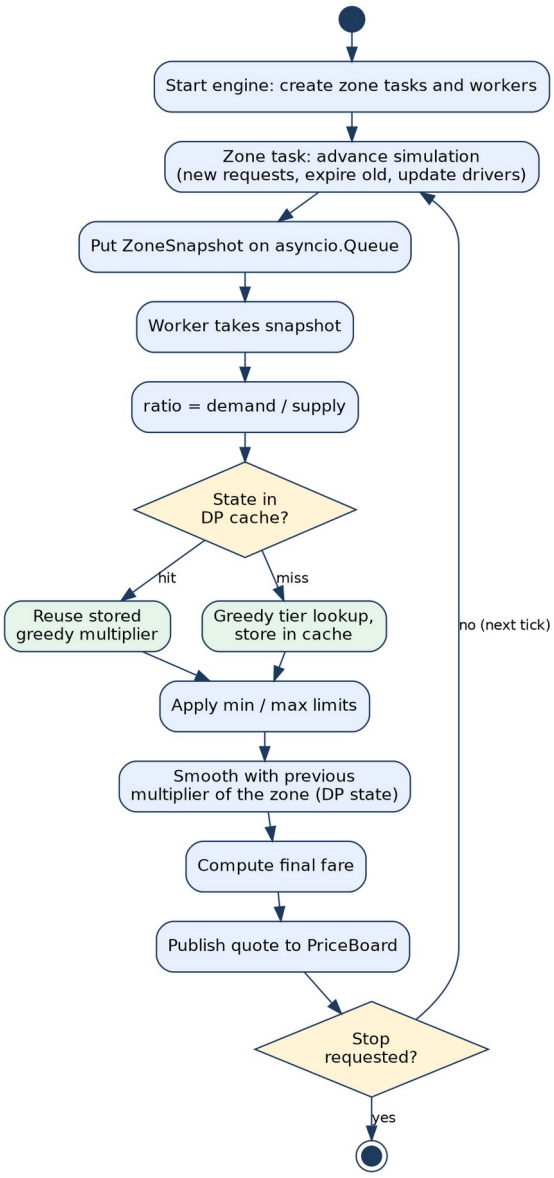


Figure 9.1: Activity diagram of the pricing flow

10. Algorithms

10.1 Dynamic Pricing Logic

The demand/supply ratio is defined as the number of pending requests divided by the number of available drivers. If there are no drivers and at least one request, the ratio is treated as infinite and the highest tier is selected; if there are no requests the ratio is zero.

```
ratio(demand, supply):
    if supply == 0:  return infinity if demand > 0 else 0
    return demand / supply
```

The ratio is converted into a multiplier by the greedy rule of Section 10.2, limited by the configured minimum and maximum and finally smoothed by the dynamic-programming recurrence of Section 10.3. The examples of the specification are reproduced exactly:

Table 10.1: Examples from the specification

Demand	Supply	Ratio	Tier	Multiplier
80	100	0.80	Balanced	1.0x
100	50	2.00	High (1.5 < r <= 2.0)	1.5x
100	20	5.00	Extreme (r > 3.0)	2.5x

10.2 Greedy Algorithm

The greedy algorithm selects the pricing tier immediately from the current ratio. It does not look at previous ratios and never revises the decision, which is exactly the definition of a greedy choice. The tiers are upper-inclusive intervals so that the boundary value 2.0 belongs to the 1.5x tier, as required by the example above.

Table 10.2: Greedy pricing tiers

Ratio	Multiplier	Label
ratio <= 1.0	1.0x	Balanced
1.0 < ratio <= 1.5	1.2x	Busy
1.5 < ratio <= 2.0	1.5x	High
2.0 < ratio <= 3.0	2.0x	Very high
ratio > 3.0	2.5x	Extreme

```
class GreedyTierStrategy(PricingStrategy):
    """Pick the first tier whose upper bound covers the current ratio."""

    def __init__(self, tiers: tuple[Tier, ...] = DEFAULT_TIERS) -> None:
        self._tiers = tiers

    def _tier(self, ratio: float) -> Tier:
        for tier in self._tiers:
            if ratio <= tier.upper:
                return tier
        return self._tiers[-1]

    def multiplier_for(self, ratio: float) -> float:
        return self._tier(ratio).multiplier

    def tier_label(self, ratio: float) -> str:
        return self._tier(ratio).label
```

Listing 10.1: Greedy tier strategy

Complexity: the tier table has a constant number of entries, so selecting a tier takes O(1) time and O(1) space. The greedy rule is simple and explainable, but it is memoryless; its weakness is that two consecutive readings with a slightly different ratio can lead to very different prices. The dynamic-programming layer corrects this.

10.3 Dynamic Programming Optimisation

The dynamic-programming layer has two parts, both implemented in surge_calculator.py.

Part 1 - memoisation of overlapping sub-problems. The sub-problem is G(zone, demand, supply), the greedy multiplier of a state. Identical states occur repeatedly (especially in small zones and at night when numbers are low), therefore the result of every sub-problem is stored in DPCache under the key (zone, demand, supply). The next request for the same key is answered by a dictionary lookup without calling the strategy. The cache is a bounded least-recently-used table with hit and miss counters, so that the benefit of memoisation can be measured and the memory use is limited.

```
class DPCache:
    """Bounded LRU memo table: ``(zone, demand, supply) -> multiplier``."""

    def __init__(self, max_size: int = 10_000) -> None:
        self._max_size = max_size
        self._table: OrderedDict[CacheKey, float] = OrderedDict()
        self.hits = 0
        self.misses = 0

    def get_or_compute(self, key: CacheKey, compute: Callable[[], float]) -> tuple[flo
at, bool]:
        """Return ``(value, was_cache_hit)``; solve the sub-problem on a miss."""
        if key in self._table:
            self._table.move_to_end(key)
            self.hits += 1
            return self._table[key], True
        value = compute()
        self._table[key] = value
        self.misses += 1
        if len(self._table) > self._max_size:
            self._table.popitem(last=False)
```

```
        return value, False
```

Listing 10.2: Memo table (DPCache)

Part 2 - state recurrence for smoothing. The multiplier published for a zone must not differ much from the multiplier published a moment ago. Let G[t] be the greedy target (after limits) and M[t-1] the previous multiplier of the same zone, which is stored in a table with one entry per zone. The new multiplier is defined by the recurrence:

```
M[t] = clamp( M[t-1] + clamp( alpha * (G[t] - M[t-1]), -step, +step ), min, max )

alpha : smoothing factor (default 0.5; 1.0 disables smoothing)
step  : maximum change per tick (default 0.5)
M[0]  = G[0]   (cold start)
```

The value M[t] depends only on M[t-1] and G[t], so the table of states is filled from the first tick onwards and every entry is reused for the next one. This is the bottom-up form of dynamic programming. The alpha term moves the price part of the way towards the target (exponential smoothing), and the step term guarantees that no single tick changes the price by more than 0.5x. The result is rounded to the nearest 0.05x, because riders should see understandable numbers. If the target is within one rounding unit of the previous value, the target is adopted directly so that the price converges exactly.

```
    def calculate(self, zone: str, demand: int, supply: int) -> SurgeResult:
        ratio = demand_supply_ratio(demand, supply)

        # (1) Memoised greedy decision.
        raw, hit = self.cache.get_or_compute(
            (zone, demand, supply), lambda: self.strategy.multiplier_for(ratio))
        target = self._clamp(raw)

        # (2) DP recurrence over the previous state of this zone.
        previous = self._state.get(zone)
        if previous is None:
            final = target                                  # cold start
        elif abs(target - previous) < self.config.rounding:
            final = target
        else:
            step = self.config.smoothing_alpha * (target - previous)
            step = max(-self.config.max_step, min(self.config.max_step, step))
            final = self._clamp(self._round(previous + step))
        self._state[zone] = final

        return SurgeResult(zone, demand, supply, ratio, target, final,
                           self.strategy.tier_label(ratio), hit)
```

Listing 10.3: SurgeCalculator.calculate

Table 10.3: Worked example of smoothing (alpha = 0.5, step = 0.5; ratio jumps from 0.8 to 5.0)

Tick	Greedy G[t]	Previous M[t-1]	Step applied	Published M[t]
0	1.00	-	-	1.00
1	2.50	1.00	0.50 (capped from 0.75)	1.50

Tick	Greedy G[t]	Previous M[t-1]	Step applied	Published M[t]
2	2.50	1.50	0.50 (capped from 0.50)	2.00
3	2.50	2.00	0.25	2.25
4	2.50	2.25	0.125	2.40
5	2.50	2.40	0.05	2.45
6	2.50	2.45	target within 0.05	2.50

Complexity: a cache hit costs O(1) average time; a miss costs O(1) for the greedy lookup plus O(1) insertion; the smoothing step is O(1). With the LRU bound C the memory is O(C) for the cache and O(Z) for the state table of Z zones. Without the DP layer the calculator would need no memory at all, but would not be able to remove sudden fluctuations; the trade-off is a small, bounded amount of memory.

Remark on honesty of the technique: the greedy value is a pure function of the ratio, so for a very small tier table the memoisation saves little time. In this project the memo table is mainly a demonstration of memoisation and a source of statistics (hit rate); the practical benefit comes from the smoothing recurrence. Measured hit rates are reported in Chapter 14.

10.4 AsyncIO Concurrency

The engine uses the following asyncio constructs:

- asyncio.create_task() starts one producer task per zone (five tasks) and a small pool of worker tasks.
- asyncio.Queue(maxsize=N) transfers ZoneSnapshot objects from producers to workers and provides back-pressure.
- await asyncio.sleep() models the interval between measurements and lets other zones run; a random jitter of plus or minus ten percent prevents all zones from waking up at the same instant.
- asyncio.gather() waits for all producers; queue.join() waits until every queued snapshot has been priced; tasks are then cancelled and awaited so that no task is leaked.

```
    async def _worker(self, queue: asyncio.Queue) -> None:
        while True:
            snap = await queue.get()
            try:
                self.price_snapshot(snap)
            finally:
                queue.task_done()

    async def run(self, ticks: int | None = None, interval: float | None = None) -> di
ct[str, int]:
        """Run until *ticks* snapshots per zone were produced (or stop())."""
        if interval is not None:
            self.control.interval = interval
        queue: asyncio.Queue = asyncio.Queue(maxsize=self._queue_size)
        producers = [asyncio.create_task(self.simulator.run_zone(z, queue, self.contro
l, ticks),
                                         name=f"zone-{z.name}") for z in self.zones]
        consumers = [asyncio.create_task(self._worker(queue), name=f"worker-{i}")
                     for i in range(self._workers)]
        try:
            counts = await asyncio.gather(*producers)
            await queue.join()                        # drain pending snapshots
        finally:
            for task in producers + consumers:
                task.cancel()
            await asyncio.gather(*producers, *consumers, return_exceptions=True)
        return {z.name: n for z, n in zip(self.zones, counts)}
```

Listing 10.4: Producer-consumer pipeline in PricingEngine

Because the zone tasks run concurrently, one tick of five zones with a 0.1 second interval takes about 0.1 seconds instead of 0.5 seconds. In the automated test five zones and five ticks completed in 0.53 seconds against 2.5 seconds for sequential processing.

10.5 Simulation Model

DemandSimulator creates a realistic pattern without real data. Demand of a zone at tick t is base_demand x (1 + 0.55 sin(2 pi (t / period + phase))), multiplied by an event factor between 1.8 and 2.6 during random events (probability 0.03 per tick, lasting 4 to 9 ticks), plus Gaussian noise. Ride requests stay pending for three ticks. Supply follows a smaller wave that lags behind demand by three ticks, so that demand outruns supply during peaks - exactly the situation in which surge pricing is needed. Each zone has its own random generator seeded with the global seed plus the CRC-32 of the zone name, so results are reproducible whatever the order in which tasks are scheduled.

11. Implementation Details

11.1 Project Structure

```
surge-pricing-engine/
|-- app/
|   |-- __init__.py
|   |-- models.py            RideRequest, Driver, Zone, ZoneProfile, ZoneSnapshot
|   |-- simulator.py         DemandSimulator, ZoneManager, SimulationControl
|   |-- surge_calculator.py  Tier, SurgeConfig, PricingStrategy, GreedyTierStrategy,
|   |                        DPCache, SurgeCalculator
|   |-- pricing_engine.py    FareConfig, FareCalculator, PriceBoard, PricingEngine
|   |-- dashboard.py         Streamlit dashboard
|   `-- main.py              command-line simulator
|-- tests/                   test_pricing.py, test_surge.py, test_concurrency.py
|-- docs/                    Architecture.md, Design.md, UserGuide.md, ...
|-- run.py   README.md   requirements.txt   .gitignore
```

Listing 11.1: Folder structure

11.2 Module Descriptions

Table 11.1: Modules

Module	Contents and role
models.py	Passive domain objects. Zone encapsulates drivers and requests and offers set_available_drivers() to move random drivers between available, on-trip and offline states, add_request(), expire_requests() and snapshot().
simulator.py	DemandSimulator.step() advances a zone by one tick; run_zone() is the async producer. ZoneManager builds the five default zones. SimulationControl holds pause, stop and interval flags in threading.Event objects.
surge_calculator.py	Configuration with validation, strategy classes, the DP cache and the calculator. SurgeConfig raises ValueError for invalid limits, alpha, step or tier tables.
pricing_engine.py	FareCalculator, PriceBoard and PricingEngine. price_snapshot() is the synchronous pricing section; run() creates the tasks; quote_trip() prices an arbitrary trip with the current multiplier of a zone.
dashboard.py	Streamlit user interface. EngineRunner starts the asyncio loop in a daemon thread and is cached with st.cache_resource so that it survives page refreshes.
main.py / run.py	Command-line interface and launcher with the commands simulate, dashboard and test, which behave the same on Windows, Linux and macOS.

11.3 Important Design Decisions

- The cache stores the un-clamped greedy multiplier so that the minimum and maximum limits can be changed from the dashboard at run time without invalidating the cache. The smoothing state is deliberately not part of the cache key because it depends on history.
- Tiers are data (Tier dataclass) and not if-else code, which makes the rule table configurable and easy to test.
- Immutable (frozen) dataclasses are used for snapshots, quotes and configurations because they are shared between tasks and threads.

- The engine never blocks the event loop: all waiting is done with await, and the only blocking primitive (a lock in PriceBoard) is held for a few microseconds.
- Console output uses UTF-8 so that the rupee sign is displayed correctly on Windows terminals.

11.4 Dashboard Implementation

The dashboard is a single Streamlit script. A cached EngineRunner object starts PricingEngine.run() in a background thread. The page body is a function decorated with st.fragment(run_every=...), which causes only that part of the page to be redrawn at the selected interval, so the sidebar widgets are not reset. The function reads the latest quote, the history and the cache statistics from the PriceBoard and draws six key figures and four charts with Plotly. Sidebar controls modify the live engine objects: the tariff of FareCalculator, the maximum multiplier and smoothing switch of SurgeCalculator, and the SimulationControl interval and pause flag.

12. Screenshots

This chapter shows screenshots of the running dashboard (python run.py dashboard). The terminal-output figure is a placeholder: run python run.py simulate and paste a screenshot of the console in place of the grey box.

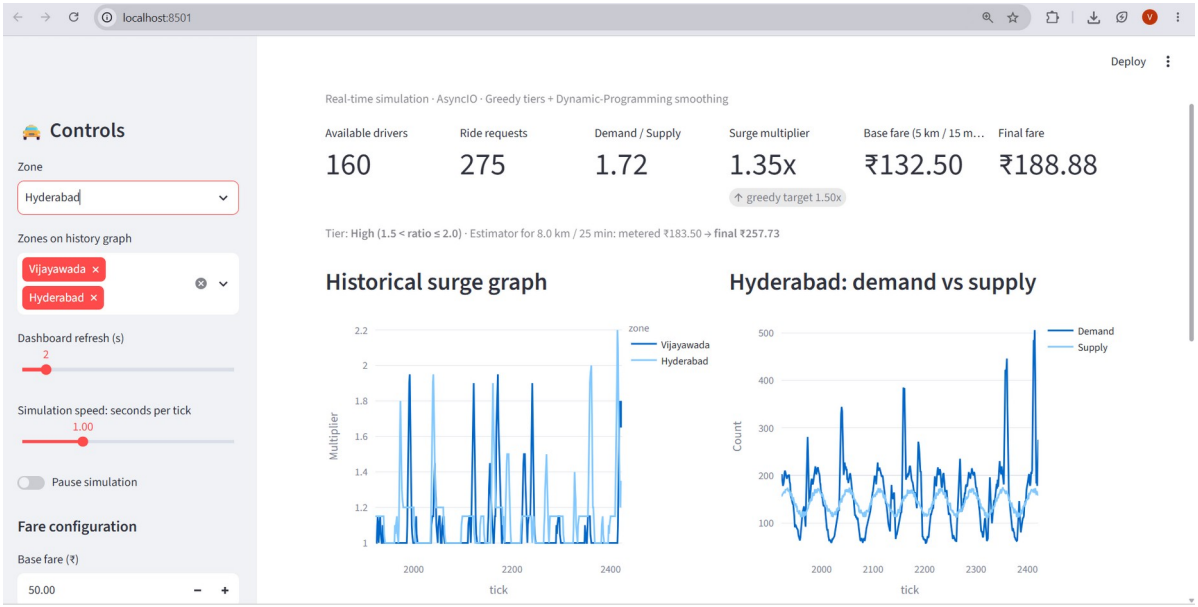


Figure 12.1: Dashboard home with key figures

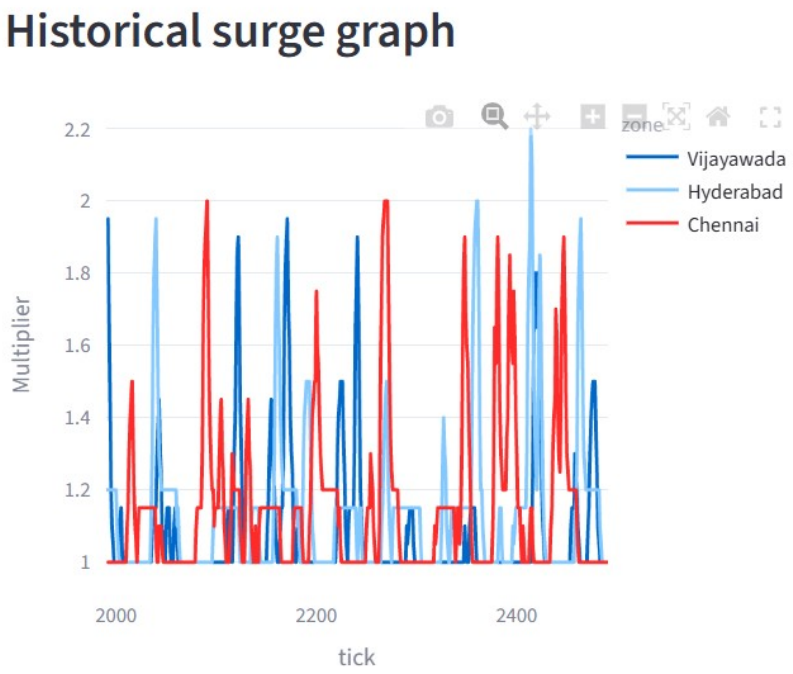


Figure 12.2: Historical surge graph

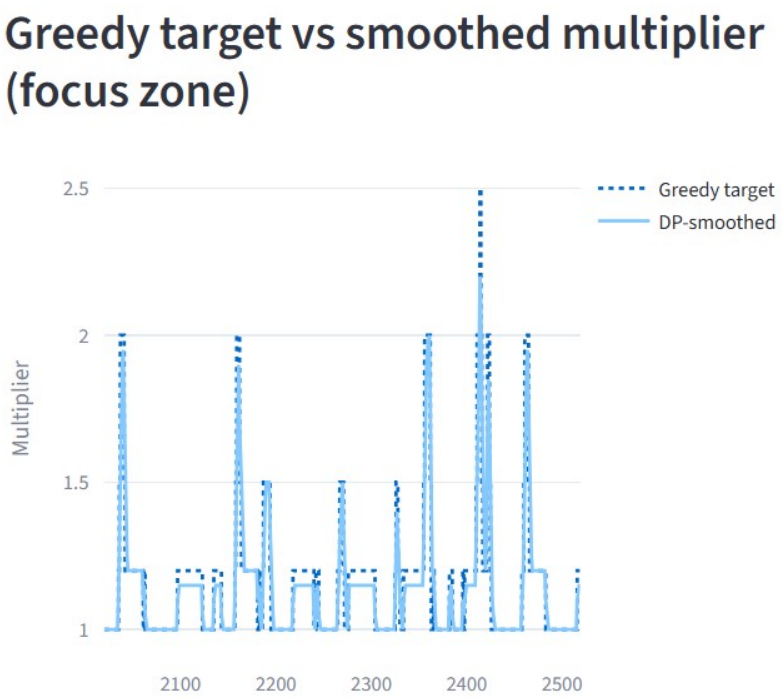


Figure 12.3: Greedy target versus DP-smoothed multiplier

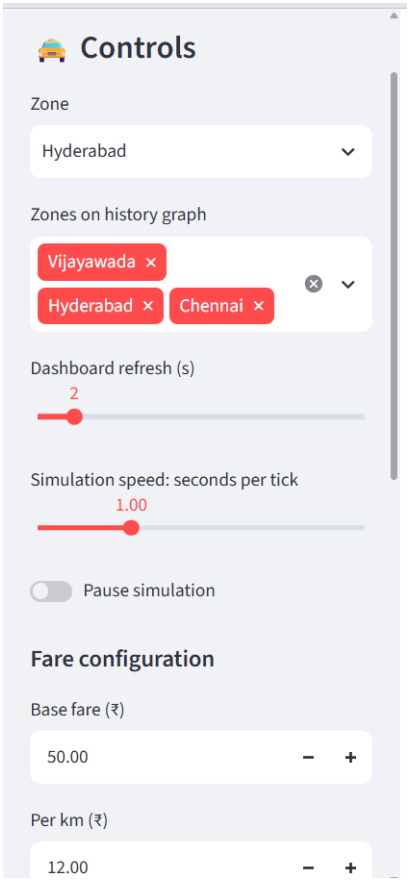
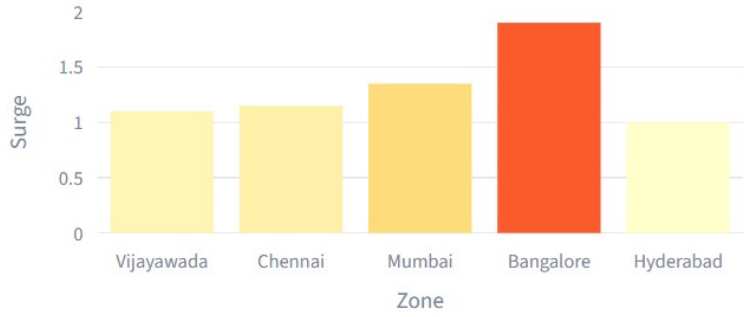


Figure 12.4: Sidebar controls and trip estimator

All zones right now



Zone	Drivers	Requests	Ratio	Surge	Final fare (₹)
Vijayawada	32	40	1.25	1.1	155.75
Chennai	132	170	1.29	1.15	162.38
Mumbai	200	294	1.47	1.35	188.88
Bangalore	164	447	2.73	1.9	261.75
Hyderabad	125	101	0.81	1	142.5

Figure 12.5: All zones right now (bar chart and table)

None

Ride-Hailing Surge Pricing Engine

Real-time simulation · AsyncIO · Greedy tiers + Dynamic-Programming smoothing

Available drivers	Ride requests	Demand / Supply	Surge multiplier	Base fare (5 km / 15 m…	Final fare
122	82	0.67	1.00x	₹132.50	₹142.50

↑ greedy target 1.00x

Tier: **Balanced (ratio ≤ 1.0)** · Estimator for 8.0 km / 25 min: metered ₹183.50 → **final ₹193.50**

Figure 12.6: Terminal simulation output

13. Test Cases

The project is verified by 45 Pytest test functions in three files. Parametrised tests expand them to 68 individual test cases. The concurrency tests use asyncio.run() directly, so no additional pytest plugin is required.

Table 13.1: Test summary

File	Tests	Coverage
test_surge.py	21	Ratio calculation and zero supply; every tier and boundary (0.99, 1.0, 1.01, 1.5, 1.51, 2.0, 2.01, 3.0, 3.01, infinity); the three specification examples; cache hit, miss, zone-specific key and LRU eviction; smoothing up and down, convergence and zone independence; minimum and maximum limits; invalid configuration; reset.
test_pricing.py	17	Fare with and without surge; minimum fare; configurable base fare; invalid input; zone supply and demand; request expiry; driver states; simulator reproducibility; default zones; engine quote, listener and bounded history.
test_concurrency.py	7	All five zones processed for every tick; queue drained; per-zone ordering with four workers; wall-clock proof of concurrency; determinism with the same seed; stop; pause and resume; control flags.

Table 13.2: Selected test cases

No.	Test case	Expected result	Status
1	Demand 80, supply 100	Multiplier 1.0x	Pass
2	Demand 100, supply 50 (ratio 2.0)	Multiplier 1.5x (upper-inclusive tier)	Pass
3	Demand 100, supply 20	Multiplier 2.5x	Pass
4	Ratio 3.01 and infinite ratio	2.5x	Pass
5	Same state calculated twice	First MISS, second HIT; hits = 1, misses = 1	Pass
6	Same numbers in two zones	Second zone is a cache MISS	Pass
7	Cache with size 2 and 3 inserts	Oldest entry evicted	Pass
8	Ratio jumps from 0.8 to 5.0	Published value rises by at most 0.5	Pass
9	Repeated high demand	Multiplier rises monotonically to 2.5x	Pass
10	Maximum limit 2.0 and demand 100, supply 10	Multiplier 2.0x	Pass
11	5 km, 15 min, 1.0x	Final fare 142.50	Pass
12	5 km, 15 min, 2.0x	Final fare 275.00	Pass
13	0.1 km, 1 min, 1.0x	Minimum fare applies: 90.00	Pass
14	Negative distance, multiplier 0.9	ValueError	Pass
15	5 zones x 5 ticks at 0.1 s	Elapsed time below 60 % of sequential time	Pass (0.53 s vs 2.5 s)

No.	Test case	Expected result	Status
16	Stop requested during an unbounded run	All tasks end; run() returns	Pass

Note: the status column records the result of the test run performed during development of this report (68 of 68 cases passed). Re-run python run.py test on your own machine and attach the console screenshot to the report.

14. Results

The experiments below were produced by running the engine with seed 42. The long run uses 300 ticks for each of the five zones (1500 snapshots). All numbers can be reproduced with python run.py simulate --ticks 300 --seed 42 --interval 0.001 --quiet.

14.1 Sample Terminal Output

```
[tick 000] Hyderabad   D= 231 S= 153 ratio= 1.51 greedy=1.50x final=1.50x fare=Rs 208.
75 cache=MISS
[tick 001] Hyderabad   D= 313 S= 158 ratio= 1.98 greedy=1.50x final=1.50x fare=Rs 208.
75 cache=MISS
[tick 002] Hyderabad   D= 377 S= 156 ratio= 2.42 greedy=2.00x final=1.75x fare=Rs 241.
88 cache=MISS
[tick 007] Hyderabad   D= 386 S= 159 ratio= 2.43 greedy=2.00x final=2.00x fare=Rs 275.
00 cache=MISS

Zone          Demand  Supply   Ratio   Surge        Fare
Vijayawada        55      42    1.31   1.25x     Rs175.62
Hyderabad        386     159    2.43   2.00x     Rs275.00
Bangalore        264     176    1.50   1.20x     Rs169.00
Chennai          103     121    0.85   1.00x     Rs142.50
Mumbai           117     144    0.81   1.00x     Rs142.50
```

Listing 14.1: Excerpt of simulator output (8 ticks)

At tick 002 Hyderabad shows the effect of smoothing: the greedy rule already asks for 2.0x, but the published multiplier is 1.75x and reaches 2.0x only at the end of the run.

14.2 Greedy Target versus Smoothed Multiplier

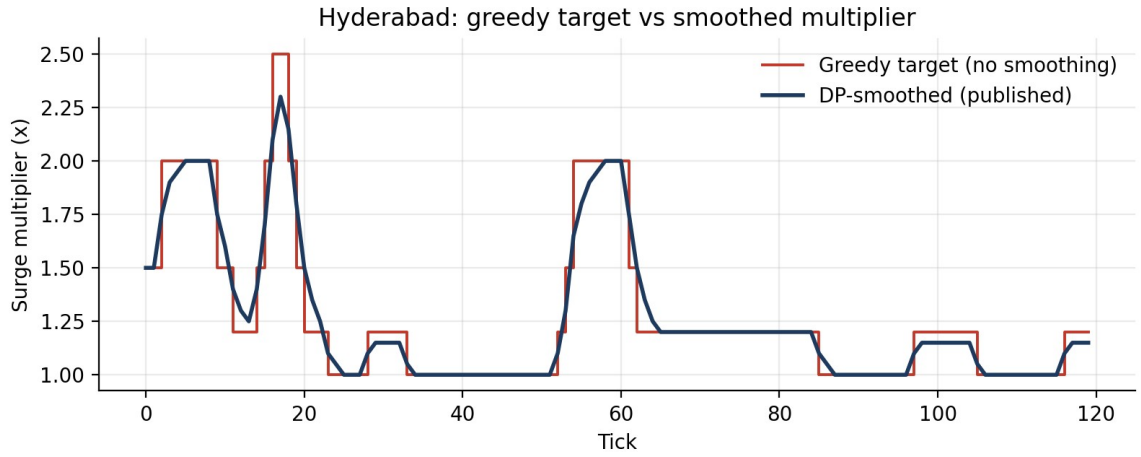


Figure 14.1: Greedy target and DP-smoothed multiplier for Hyderabad (120 ticks)

The stepped red line is the raw greedy decision and the blue line is the published multiplier. Sharp one-tick jumps of the red line are absorbed by the blue line, which follows the same trend with a short delay.

14.3 Demand and Supply

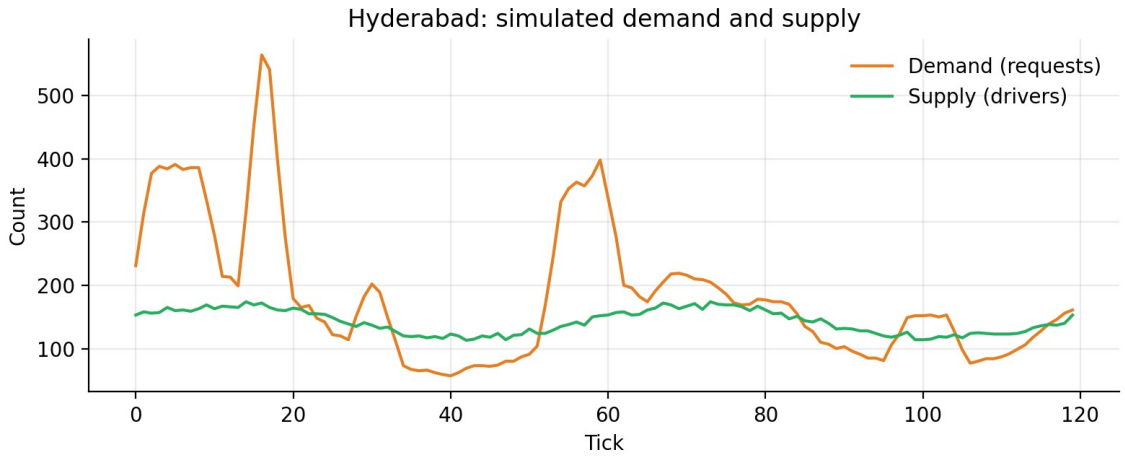


Figure 14.2: Simulated demand and supply in Hyderabad

14.4 Concurrent Zones

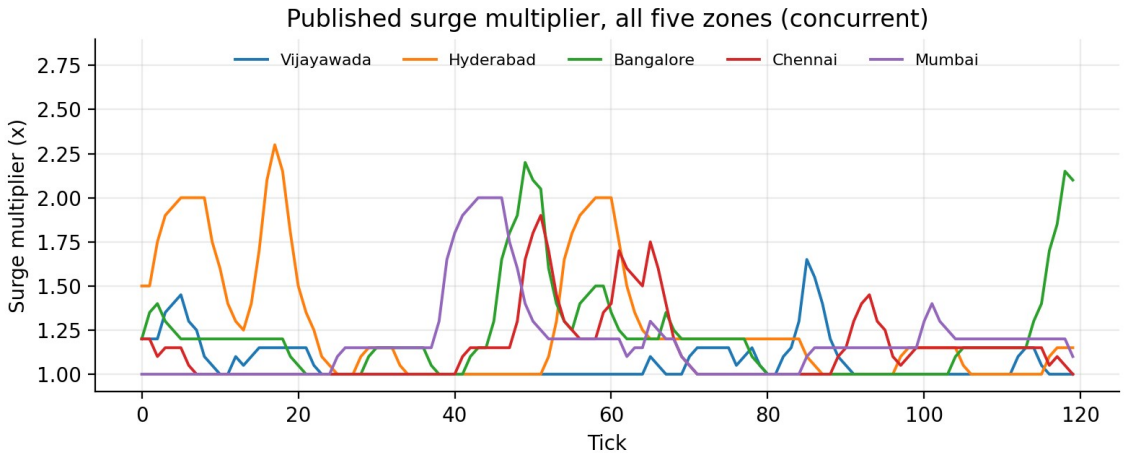


Figure 14.3: Published multipliers of the five zones

All five zones are priced in the same run, each reacting to its own demand pattern. Mumbai and Hyderabad reach higher multipliers during their peaks, while the small Vijayawada zone is calmer.

14.5 Price Stability

Table 14.1: Effect of smoothing (300 ticks per zone, seed 42)

Zone	Mean ratio	Mean greedy	Mean published	Mean abs. change greedy	Mean abs. change smoothed	Jumps >= 0.5x (greedy / smoothed)
Vijayawada	0.91	1.12	1.12	0.049	0.038	11 / 0
Hyderabad	1.16	1.22	1.21	0.057	0.048	18 / 0
Bangalore	1.18	1.19	1.18	0.050	0.037	10 / 0
Chennai	1.00	1.14	1.12	0.047	0.032	8 / 0

Zone	Mean ratio	Mean greedy	Mean published	Mean abs. change greedy	Mean abs. change smoothed	Jumps >= 0.5x (greedy / smoothed)
Mumbai	1.27	1.24	1.22	0.049	0.042	12 / 0
All zones	-	-	-	0.0505	0.0395	59 / 0

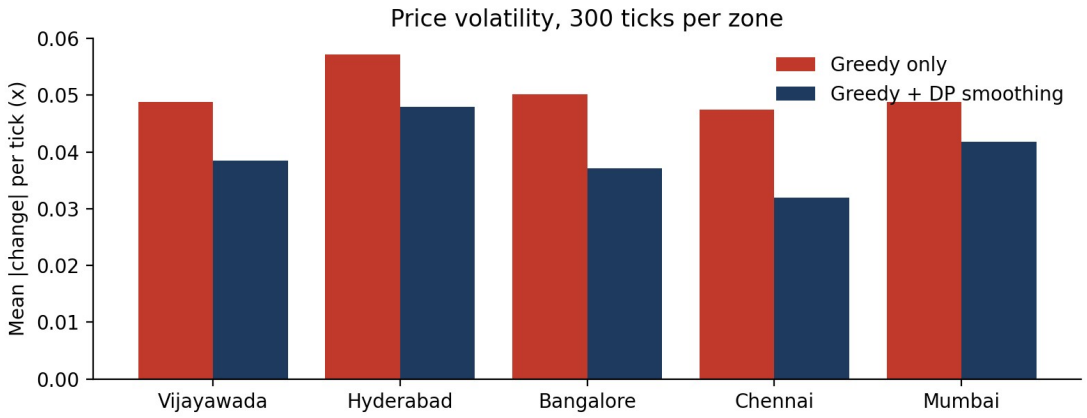


Figure 14.4: Mean absolute multiplier change per tick

Smoothing reduced the average tick-to-tick change of the multiplier by about 22 percent (0.0505 to 0.0395), removed all 59 jumps of 0.5x or more, and reduced the largest single-tick change from 0.80x to 0.45x. The mean multiplier is practically unchanged (for example 1.24 against 1.22 in Mumbai), which shows that smoothing delays the price changes but does not systematically raise or lower prices. The highest published multiplier in the run was 2.3x (Hyderabad and Mumbai), corresponding to a fare of 314.75 for the reference trip.

14.6 Dynamic Programming Cache Statistics

Table 14.2: Cache statistics

Run	Hits	Misses	Hit rate
8 ticks x 5 zones (40 states)	0	40	0.0 %
60 ticks x 5 zones (300 states)	6	294	2.0 %
300 ticks x 5 zones (1500 states)	131	1369	8.7 %
600 ticks x 5 zones (3000 states)	416	2584	13.9 %

The hit rate grows with the length of the run because more states repeat. The numbers are modest because the exact pair (demand, supply) of a large zone repeats rarely; a production system could quantise demand and supply into buckets to raise the hit rate at the cost of precision. This limitation is stated openly: in this project the stabilising effect of the DP recurrence is more valuable than the speed-up from caching.

15. Advantages

- Transparent and explainable: the pricing tiers can be read in one table.
- Stable prices: the recurrence limits the speed of change and removes abrupt spikes, which improves customer trust and affordability.
- Responsive: the greedy decision reacts immediately and the smoothed value converges to the target within a few ticks.
- Concurrent: all zones are priced simultaneously and a slow zone does not delay the others.
- Configurable: tiers, limits, smoothing, tariff and simulation speed can be changed without editing the algorithm.
- Measurable: cache statistics and the greedy-versus-smoothed comparison are available in the dashboard.
- Testable and reproducible: seeded random generators and 68 automated test cases.
- Lightweight: no database, broker or paid service is required.

16. Future Enhancements

- Replace the simulator by a real feed (for example Kafka or WebSocket streams) and persist history in a database such as PostgreSQL or Redis.
- Use geospatial indexing (geohash or H3 hexagons) to price very small areas instead of whole cities, and smooth prices across neighbouring cells.
- Add demand forecasting with machine learning so that the price reacts to expected demand and not only to current demand.
- Quantise (demand, supply) before caching to increase the cache hit rate, or share the cache between processes with Redis.
- Introduce rider elasticity and driver response models to find the multiplier that maximises completed trips instead of using fixed tiers.
- Add fairness rules, such as caps during declared emergencies and notifications that tell riders when the price is expected to fall.
- Provide a REST API (FastAPI) for the mobile applications and add authentication and monitoring.
- Containerise the system with Docker and deploy it with horizontal scaling by sharding zones across worker processes.

17. Conclusion

The Ride-Hailing Surge Pricing Engine demonstrates how classical algorithmic techniques and modern Python concurrency can be combined into a compact and understandable pricing system. The greedy tier rule converts the current demand/supply ratio into a multiplier immediately and reproduces the examples of the specification. The dynamic-programming layer stores solved states in a memo table and uses a per-zone recurrence to smooth the price, which in the experiments removed every jump of 0.5x or more and reduced average price movement by about 22 percent without changing the average price level. AsyncIO allows five zones to be generated and priced at the same time, which was confirmed by a timing test (0.53 seconds against 2.5 seconds for sequential processing). The object-oriented design keeps the components replaceable, and the Pytest suite of 68 cases gives confidence in correctness.

The project also makes clear the limitations of the approach: demand and supply are simulated, the cache hit rate is modest when exact states rarely repeat, and a ratio-based tier table ignores rider elasticity and driver behaviour. These limitations point to the future work listed in the previous chapter. As an educational and architectural reference, however, the system fulfils all the objectives set at the start of the project.

References

[1] Banerjee, S., Riquelme, C. and Johari, R. (2015). Pricing in Ride-Share Platforms: A Queueing-Theoretic Approach. SSRN working paper.

[2] Bellman, R. (1957). Dynamic Programming. Princeton University Press.

[3] Castillo, J. C., Knoepfle, D. and Weyl, G. (2017). Surge Pricing Solves the Wild Goose Chase. Proceedings of the 2017 ACM Conference on Economics and Computation (EC '17).

[4] Chen, M. K. and Sheldon, M. (2016). Dynamic Pricing in a Labor Market: Surge Pricing and Flexible Work on the Uber Platform. Proceedings of the 2016 ACM Conference on Economics and Computation (EC '16).

[5] Cohen, P., Hahn, R., Hall, J., Levitt, S. and Metcalfe, R. (2016). Using Big Data to Estimate Consumer Surplus: The Case of Uber. NBER Working Paper No. 22627.

[6] Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. Introduction to Algorithms. MIT Press (4th edition, 2022).

[7] Gamma, E., Helm, R., Johnson, R. and Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

[8] Hall, J., Kendrick, C. and Nosko, C. (2015). The Effects of Uber's Surge Pricing: A Case Study. Uber Technologies / University of Chicago Booth School of Business.

[9] Python Software Foundation. asyncio - Asynchronous I/O. Python 3 documentation, https://docs.python.org/3/library/asyncio.html

[10] Streamlit Inc. Streamlit documentation, https://docs.streamlit.io

[11] Plotly Technologies. Plotly Python graphing library, https://plotly.com/python/

[12] Pytest developers. pytest documentation, https://docs.pytest.org